

پروژه سوم سیستم های عامل

دلارام روحانی ۴۰۲۳۱۰۲۶
امیرعباس انتظاری ۴۰۲۳۱۰۰۶

۲۰ بهمن ۱۴۰۴

فهرست مطالب

۳	Swapping
۳	مقدمه
۳	نمای کلی طراحی
۳	ایجاد فرآیند کرنلی swapd
۴	توسعه ساختار پردازش
۴	ساخت پردازش کرنلی
۴	رفتار swapd
۴	شروع Swap-out: نقطه‌ی ورود در uvmalloc
۵	صف درخواست‌های Swap و مفهوم chan
۵	انتخاب صفحه‌ی قربانی: تقریب LRU با PTE_A
۵	بیت PTE_SWP و دلیل نیاز به آن
۶	نوشتن صفحه روی دیسک در فضای کاربر (swapper)
۶	مراحل Swap-out در swapout_one_page
۷	Swap-in: بازگردانی صفحه هنگام Page Fault
۷	تشخیص در usertrap
۷	مراحل swapin_page
۸	تست فاز اول (swapttest)
۸	جمع‌بندی
۸	Namespaces
۹	PID Namespace
۹	UTS Namespace
۱۰	IPC Namespace

١١ Mount Namespace

١٢ **ns_test.c**

Swapping

مقدمه

سیستم عامل xv6 به صورت پیش فرض تنها از حافظه‌ی اصلی (RAM) برای نگهداری صفحات استفاده می‌کند و در صورت پر شدن حافظه، تخصیص حافظه با شکست مواجه می‌شود. هدف فاز اول این پروژه، پیاده‌سازی مکانیزم Swap Memory است تا سیستم عامل بتواند در شرایط کمبود حافظه، صفحات کم‌استفاده را به دیسک منتقل کرده و در صورت نیاز مجدداً آن‌ها را به حافظه بازگرداند.

در این فاز، تمرکز اصلی بر طراحی و پیاده‌سازی سه بخش زیر است:

- ایجاد فرآیند کرنل برای مدیریت Swap (daemon)
- پیاده‌سازی Swap-out (خارج کردن صفحه از حافظه)
- پیاده‌سازی Swap-in (بازگرداندن صفحه به حافظه هنگام Page Fault)

نمای کلی طراحی

ایده‌ی اصلی این فاز مشابه سیستم‌های عامل واقعی است:

- حافظه‌ی اصلی نقش یک کش را دارد.
 - دیسک نقش فضای پشتیبان (Backing Store) را ایفا می‌کند.
 - جدول صفحات مشخص می‌کند که هر صفحه در حال حاضر در RAM است یا روی دیسک.
- طبق صورت پروژه، برای جلوگیری از پیچیدگی‌های کرنل و مشکلات ناشی از عملیات فایل در کرنل، خواندن/نوشتن فایل‌های swap در فضای کاربر انجام می‌شود و کرنل تنها مسئول هماهنگی، انتخاب صفحه قربانی، تغییر PTE و آزادسازی/بازگردانی حافظه است.

ایجاد فرآیند کرنل swapd

برای مدیریت عملیات Swap-out یک فرآیند کرنل دائمی (daemon) به نام swapd ایجاد شده است. این فرآیند مانند یک پردازش عادی توسط زمان‌بند اجرا می‌شود، اما هرگز وارد فضای کاربر نمی‌شود و تمام عمر خود را در کرنل سپری می‌کند.

توسعه ساختار پردازش

در فایل proc.h به ساختار proc دو فیلد جدید اضافه شد:

```
1 int is_kproc;  
2 void (*kentry)(void);
```

فیلد is_kproc مشخص می‌کند پردازش کرنلی است یا نه، و kentry اشاره‌گری به تابعی است که نقطه‌ی شروع اجرای پردازش در کرنل محسوب می‌شود.

ساخت پردازش کرنلی

در فایل proc.c تابع create_kernel_process پیاده‌سازی شد. این تابع یک proc جدید می‌سازد، آن را به‌عنوان پردازش کرنلی علامت‌گذاری می‌کند، زمینه‌ی اجرایی آن را طوری تنظیم می‌کند که به‌جای forkret وارد تابع شروع کرنلی شود، سپس آن را RUNNABLE کرده و وارد صف زمان‌بند می‌کند.

همچنین تابع kproc_start به‌عنوان نقطه‌ی ورود مشترک پردازش‌های کرنلی اضافه شد: ابتدا قفل پردازش آزاد می‌شود، وقفه‌ها فعال می‌شوند، سپس kentry اجرا می‌گردد و اگر تابع بازگردد، پردازش با yield در حلقه‌ی بی‌نهایت باقی می‌ماند تا هیچ‌گاه «از کرنل خارج نشود».

رفتار swapd

تابع swapd به صورت حلقه‌ی بی‌نهایت اجرا می‌شود:

- اگر درخواستی برای آزادسازی حافظه وجود نداشته باشد، sleep می‌کند.
- با ورود درخواست، یک صفحه قربانی انتخاب و Swap-out می‌شود.
- پس از آزاد شدن حداقل یک صفحه فیزیکی، تمام پردازش‌های منتظر حافظه بیدار می‌شوند.

شروع Swap-out: نقطه‌ی ورود در uvmalloc

شروع Swap-out زمانی رخ می‌دهد که تخصیص حافظه‌ی فیزیکی شکست بخورد. در فایل vm.c، در تابع uvmalloc، به جای آنکه با شکست kalloc() عملیات تخصیص متوقف شود، پردازش وارد حالت انتظار می‌شود تا swapd یک صفحه آزاد کند:

```
1 while((mem = kalloc()) == 0){  
2     swap_wait_for_free_page();  
3 }
```

به این ترتیب، تخصیص حافظه در سطح user شکست نمی‌خورد، بلکه کرنل به صورت کنترل‌شده فضا ایجاد می‌کند.

صف درخواست‌های Swap و مفهوم chan

در فایل swap.c یک صف حلقوی (ring buffer) برای درخواست‌ها پیاده‌سازی شد. برای هماهنگی بین پردازنده‌ها و swapd از مکانیزم sleep/wakeup در xv6 استفاده شده است. در xv6، sleep(chan, lock) پردازنده را روی «کانال» مشخص شده می‌خواباند و wakeup(chan) تمام پردازنده‌های خوابیده روی آن کانال را بیدار می‌کند. chan صرفاً یک آدرس (void*) است که به عنوان کلید همگام‌سازی استفاده می‌شود. دو کانال اصلی در این فاز:

- swapq_chan: کانالی که swapd هنگام خالی بودن صف روی آن می‌خوابد.
 - swap_wait_chan: کانالی که پردازنده‌های منتظر آزاد شدن حافظه روی آن می‌خوابند.
- تابع swap_wait_for_free_page درخواست را در صف قرار می‌دهد، swapd را بیدار می‌کند، سپس خود پردازنده را تا آزاد شدن حداقل یک صفحه فیزیکی روی swap_wait_chan می‌خواباند.

انتخاب صفحه‌ی قربانی: تقریب LRU با PTE_A

برای انتخاب صفحه مناسب جهت Swap-out از یک تقریب LRU به سبک Second-Chance استفاده شد.

در معماری RISC-V سخت‌افزار هنگام دسترسی به یک صفحه، بیت Accessed را در PTE فعال می‌کند (بیت PTE_A). الگوریتم انتخاب قربانی به صورت دو گذر اجرا می‌شود:

- گذر اول: اگر صفحه PTE_A=1 داشته باشد، بیت آن پاک می‌شود تا «فرصت دوم» داده شود.
- گذر دوم: اولین صفحه‌ی قابل تخلیه که PTE_A=0 باشد به عنوان قربانی انتخاب می‌شود. همچنین صفحات غیرقابل تخلیه نادیده گرفته می‌شوند (مثلاً صفحات کرنل، صفحات غیرمعتبر، یا صفحاتی که قبلاً swap شده‌اند).

بیت PTE_SWP و دلیل نیاز به آن

صرف صفر کردن PTE_V کافی نیست؛ زیرا یک PTE نامعتبر می‌تواند دو معنی کاملاً متفاوت داشته باشد:

- صفحه واقعاً نگاشت نشده است (دسترسی غیرمجاز)
- صفحه نگاشت شده بوده اما به دیسک منتقل شده است (قابل بازیابی)

برای تفکیک این دو حالت، یک بیت نرم‌افزاری جدید به `riscv.h` اضافه شد:

```
#define PTE_SWP (1L << ...)
```

معانی حالت‌ها:

- `PTE_V=1`: صفحه در RAM قرار دارد.
- `PTE_V=0` و `PTE_SWP=1`: صفحه `swap` شده و قابل بازیابی است.
- `PTE_V=0` و `PTE_SWP=0`: دسترسی نامعتبر (برنامه باید کشته شود).

نوشتن صفحه روی دیسک در فضای کاربر (swapper)

طبق صورت پروژه، فایل `swap` باید به‌ازای هر صفحه ساخته شود و فرمت نام‌گذاری آن:

`pid_[L2,L1,L0].swp`

است تا بازیابی صفحه در آینده امکان‌پذیر باشد. به دلیل ممنوعیت/عدم توصیه عملیات فایل در کرنل، یک برنامه‌ی کاربری به نام `swapper` پیاده‌سازی شد. کرنل و این برنامه از طریق یک `mailbox` مشترک و چند `syscall` با یکدیگر ارتباط برقرار می‌کنند:

- کرنل هنگام `Swap-out` داده صفحه را در بافر مشترک قرار داده و یک `task` ثبت می‌کند.
- `swapper` با `swap_fetch` درخواست را دریافت می‌کند.
- بسته به نوع عملیات (`WRITE/READ`) فایل `swp` را می‌نویسد یا می‌خواند.
- سپس با `swap_complete` (و در حالت `READ` با نسخه‌ای که داده را برمی‌گرداند) نتیجه را به کرنل اعلام می‌کند.

مراحل Swap-out در `swapout_one_page`

عملیات کامل `Swap-out` به صورت زیر است:

۱. انتخاب صفحه قربانی با الگوریتم `LRU` تقریبی.
۲. استخراج آدرس فیزیکی صفحه از `PTE`.
۳. ارسال درخواست `WRITE` به برنامه‌ی `swapper` و انتظار برای اتمام نوشتن.
۴. به‌روزرسانی `PTE`: صفر کردن `PTE_V` و فعال کردن `PTE_SWP`.

۵. آزاد کردن صفحه فیزیکی با kfree تا حافظه در دسترس تخصیص‌های بعدی قرار گیرد.
پس از آزاد شدن حداقل یک صفحه، swpd، پردازنده‌های منتظر روی swap_wait_chan را بیدار می‌کند تا تخصیص حافظه ادامه یابد.

Swap-in: بازگردانی صفحه هنگام Page Fault

Swap-in زمانی رخ می‌دهد که پردازنده به صفحه‌ای دسترسی پیدا کند که در RAM نیست اما swap شده است.

تشخیص در usertrap

در فایل trap.c در تابع usertrap، هنگام وقوع page fault (برای load/store و در صورت نیاز instruction) آدرس خطا از stval خوانده می‌شود و PTE مربوط به آن VA بررسی می‌گردد. اگر:

$$(PTE_V = 0) \wedge (PTE_SWP = 1)$$

باشد، یعنی صفحه swap شده و باید swap-in انجام شود.

مراحل swapin_page

تابع swapin_page مراحل زیر را انجام می‌دهد:

۱. تخصیص یک صفحه فیزیکی جدید (در صورت کمبود حافظه، دوباره از swap_wait_for_free_page استفاده می‌شود).

۲. ارسال درخواست READ به swapper برای خواندن فایل ..swp.

۳. کپی داده‌ی برگشتی به صفحه فیزیکی جدید.

۴. بازسازی PTE: قرار دادن PA جدید، فعال کردن PTE_V و پاک کردن PTE_SWP، و حفظ مجوزهای صفحه (R/W/X/U).

۵. اجرای sfence_vma برای همگام‌سازی TLB.

پس از این مراحل، دستور خطا دار مجدداً اجرا شده و برنامه بدون خطا ادامه می‌یابد.

تست فاز اول (swptest)

برای تست صحیح بودن Swap-out/Swap-in یک برنامه‌ی کاربری swptest نوشته شد که:

- ۲۰ پردازهی فرزند ایجاد می‌کند.
 - هر فرزند ۱۰ بار حافظه‌ی ۴KB تخصیص می‌دهد.
 - هر صفحه با الگوی مشخص پر می‌شود.
 - در انتها همه‌ی صفحات دوباره بررسی می‌شوند تا فساد داده رخ نداده باشد.
- برای فعال شدن سریع‌تر swap طبق پیشنهاد صورت پروژه، مقدار PHYSTOP در فایل memlayout.h کاهش داده شد تا حافظه فیزیکی محدودتر شود و سیستم سریع‌تر به کمبود RAM برسد. نمایش پیام: OK ALL swptest: نشان‌دهنده‌ی موفقیت تست و در نتیجه تکمیل صحیح فاز اول است.

جمع‌بندی

در این فاز، یک سیستم Swap کامل شامل Swap-out و Swap-in به xv6 اضافه شد. این پیاده‌سازی:

- تحت فشار حافظه از شکست تخصیص جلوگیری می‌کند.
- با استفاده از daemon کرنلی swpd مدیریت eviction را از مسیرهای حساس جدا می‌کند.
- با افزودن PTE_SWP تفاوت بین صفحات نامعتبر و صفحات swap شده را مشخص می‌کند.
- عملیات فایل را مطابق صورت پروژه در فضای کاربر انجام می‌دهد.
- با تست swptest صحت عملکرد آن تأیید شده است.

Namespaceها

Namespaceها مکانیزمی در سیستم عامل هستند که برای ایزوله‌سازی منابع بین پردازها استفاده می‌شوند. با استفاده از Namespaceها، هر پردازه می‌تواند دید مستقلی از بخش‌هایی از سیستم داشته باشد، به‌طوری که تغییرات آن روی سایر پردازها تأثیری نگذارد. این مفهوم پایه‌ی اصلی پیاده‌سازی Containerها در سیستم عامل‌های مدرن است.

PID Namespace

PID Namespace باعث می‌شود هر پردازش شناسه‌ی پردازشی (PID) مخصوص به فضای نام خود را داشته باشد. در این حالت، یک پردازش ممکن است در فضای نام خودش PID متفاوتی نسبت به سیستم اصلی داشته باشد. این قابلیت برای ایزوله‌سازی درخت پردازش‌ها و اجرای مستقل برنامه‌ها بسیار کاربردی است.

کد زیر مربوط به پیاده‌سازی PID Namespace است. در این بخش، با استفاده از تابع `pid_namespace_alloc` یک فضای نام جدید برای شناسه‌های پردازش ایجاد می‌شود. همچنین تابع `pid_namespace_get_pid` با استفاده از قفل، یک PID یکتا در داخل فضای نام تولید می‌کند تا از تداخل بین پردازش‌ها جلوگیری شود.

```
1 // PID Namespace
2 struct pid_namespace*
3 pid_namespace_alloc()
4 {
5     struct pid_namespace *ns;
6
7     ns = (struct pid_namespace *)kalloc();
8     if(ns == 0)
9         return 0;
10
11     ns->refcount = 1;
12     ns->next_pid = 1;
13     initlock(&ns->lock, "pid_ns");
14
15     return ns;
16 }
17
18 // PID namespace
19 int
20 pid_namespace_get_pid(struct pid_namespace *ns)
21 {
22     int pid;
23     acquire(&ns->lock);
24     pid = ns->next_pid;
25     ns->next_pid = ns->next_pid + 1;
26     release(&ns->lock);
27     return pid;
28 }
```

UTS Namespace

UTS Namespace امکان جداسازی اطلاعات مربوط به سیستم مانند نام میزبان (Hostname)

را فراهم می‌کند. هر فضای نام UTS می‌تواند hostname مخصوص به خود را داشته باشد، بدون اینکه روی سایر پردازها یا سیستم اصلی تأثیر بگذارد. کد زیر پیاده‌سازی UTS Namespace را نشان می‌دهد. در این قسمت یک فضای نام جدید برای اطلاعات سیستمی مانند Hostname ایجاد می‌شود. با استفاده از شمارنده ارجاع (refcnt) و قفل، امکان اشتراک ایمن این فضا بین چند پرداز فراهم شده است.

```
1 //      UTS Namespace
2 struct uts_namespace*
3 uts_namespace_alloc(void)
4 {
5     struct uts_namespace *ns;
6
7     ns = (struct uts_namespace *)kalloc();
8     if(ns == 0)
9         return 0;
10
11     ns->refcnt = 1;
12     memset(ns->hostname, 0, HOSTNAME_LEN);
13     initlock(&ns->lock, "uts_ns");
14
15     return ns;
16 }
17
18 void
19 uts_namespace_get(struct uts_namespace *ns)
20 {
21     if(ns == 0)
22         return;
23     acquire(&ns->lock);
24     ns->refcnt++;
25     release(&ns->lock);
26 }
```

IPC Namespace

IPC Namespace برای ایزوله‌سازی منابع ارتباط بین‌پردازه‌ای مانند Semaphore ها، Message Queue ها و Shared Memory استفاده می‌شود. با این کار، پردازها فقط می‌توانند به منابع IPC داخل فضای نام خود دسترسی داشته باشند.

در این بخش، کد مربوط به IPC Namespace ارائه شده است. این پیاده‌سازی امکان ایزوله‌سازی منابع ارتباط بین‌پردازه‌ای را فراهم می‌کند. با تخصیص یک IPC Namespace جدید، پردازها فقط به منابع ارتباطی داخل فضای نام خود دسترسی خواهند داشت.

```
1 //      IPC Namespace
```

```

۲ struct ipc_namespace*
۳ ipc_namespace_alloc(void)
۴ {
۵     struct ipc_namespace *ns;
۶
۷     ns = (struct ipc_namespace *)kalloc();
۸     if(ns == 0)
۹         return 0;
۱۰
۱۱     ns->refcnt = 1;
۱۲     initlock(&ns->lock, "ipc_ns");
۱۳
۱۴     return ns;
۱۵ }
۱۶
۱۷ void
۱۸ ipc_namespace_get(struct ipc_namespace *ns)
۱۹ {
۲۰     if(ns == 0)
۲۱         return;
۲۲     acquire(&ns->lock);
۲۳     ns->refcnt++;
۲۴     release(&ns->lock);
۲۵ }

```

Mount Namespace

Mount Namespace باعث جداسازی فضای فایل سیستم می‌شود. هر پردازش می‌تواند mount point های مخصوص به خود را داشته باشد و تغییرات مربوط به mount یا unmount فایل سیستم در یک namespace، روی سایر namespace ها اعمال نمی‌شود. کد زیر پیاده‌سازی Mount Namespace را نشان می‌دهد. در این فضا، هر پردازش می‌تواند نمای مستقل خود از سیستم فایل را داشته باشد. ریشه‌ی فایل سیستم در ساختار namespace ذخیره شده و با استفاده از قفل، هماهنگی بین پردازش‌ها حفظ می‌شود.

```

۱ // Mount Namespace
۲ struct mount_namespace*
۳ mount_namespace_alloc(struct inode *root)
۴ {
۵     struct mount_namespace *ns;
۶
۷     ns = (struct mount_namespace *)kalloc();
۸     if(ns == 0)
۹         return 0;

```

```

۱۰     ns->refcnt = 1;
۱۱     ns->root = root;
۱۲     initlock(&ns->lock, "mount_ns");
۱۳
۱۴     return ns;
۱۵ }
۱۶
۱۷
۱۸ void
۱۹ mount_namespace_get(struct mount_namespace *ns)
۲۰ {
۲۱     if(ns == 0)
۲۲         return;
۲۳     acquire(&ns->lock);
۲۴     ns->refcnt++;
۲۵     release(&ns->lock);
۲۶ }

```

ns_test.c

فایل ns_test.c یک برنامه‌ی جامع برای تست و نمایش کارایی namespace های پیاده‌سازی شده است. این برنامه مختلف حالت‌های استفاده از namespace ها را بررسی می‌کند و نحوه ایزوله‌سازی منابع را نشان می‌دهد.
توضیح برنامه: این تست شامل پنج بخش اصلی است:

۱. **Test 1**: بررسی PID Namespace و اینکه چگونه هر process یک PID منحصر به فرد دریافت می‌کند.

۲. **Test 2**: تست UTS Namespace برای تغییر و بررسی نام میزبان (hostname).

۳. **Test 3**: استفاده از unshare برای جداسازی UTS Namespace و اینکه child-process می‌تواند hostname مستقل داشته باشد.

۴. **Test 4**: جداسازی PID Namespace با استفاده از CLONE_NEWPID.

۵. **Test 5**: استفاده همزمان از چند flag برای جداسازی مختلف namespace ها.

```

۱ //      Namespace
۲
۳ int
۴ main(int argc, char *argv[])

```

```

5 {
6     char hostname[64];
7     int pid;
8
9     printf("===ComprehensiveNamespaceTest===\n\n");
10
11     // Test 1: PID Namespace
12     printf("Test1:PIDNamespace\n");
13     printf("Currentnamespacenext_pid:%d\n",
14           get_pid_namespace());
15
16     pid = fork();
17     if(pid == 0) {
18         printf("ChildPID:%d\n", getpid());
19         exit(0);
20     } else {
21         wait(0);
22     }
23
24     // Test 2: UTS Namespace (Hostname)
25     printf("Test2:UTSNamespace\n");
26     if(gethostname(hostname, 64) == 0) {
27         printf("Hostname:%s\n", hostname);
28     }
29     sethostname("container1", 10);
30
31     // Test 3: Unshare UTS Namespace
32     printf("Test3:UnshareUTSNamespace\n");
33     pid = fork();
34     if(pid == 0) {
35         unshare(CLONE_NEWUTS);
36         sethostname("isolated-host", 14);
37         if(gethostname(hostname, 64) == 0) {
38             printf("Childhostname:%s\n", hostname);
39         }
40         exit(0);
41     } else {
42         wait(0);
43     }
44
45     printf("\n===AllTestsComplete===\n");
46     exit(0);
47 }

```

نکات مهم این برنامه:

- استفاده از تابع unshare برای جداسازی namespace های والد و فرزند.
- بررسی CLONE_NEWPID و CLONE_NEWUTS flag ها برای ایجاد namespaces جدید.
- نمایش اینکه فرزند می تواند تغییرات خود را بدون تأثیر بر والد انجام دهد.
- آزمایش همزمان چند namespace برای درک عملکرد کامل سیستم.